

15

Dasturlash tilidagi kompozitsiya muammosi Java

15.1. MUAMMO TAVSIFI

Ma'lumki, Java dasturlash tilida ko'rsatkich (pointer) tushunchasi mavjud emas. Ya'ni, barcha primitiv bo'lmagan turlar (massivlar, satrlar, istalgan sinf obyektlari) ko'rsatkich sifatida amalga oshirilgan, ammo ular o'zlarini ko'rsatkich ekanliklarini ko'rsatmaydilar. Shunday qilib, yaratilgan satrlarga, massivlarga va sinf instantsiyalari sifatida yaratilgan barcha obyektlarga faqat ko'rsatkichlar orqali murojaat qilish mumkin. Maydonlar, o'zgaruvchilar, metod parametrlari va yaratilgan anonim miqdorlar (ular uchun ajratilgan xotirada) yoki primitiv qiymatni, yoki ular murojaat qiladigan tuzilmaning manzilini o'z ichiga oladi.

Shunday qilib, Java ko'rsatkichlar bilan bog'liq keskin muammolarni hal qiladi. "Dangling pointer" (ishonchsiz ko'rsatkich) muammosi dasturlash tillarida keng tarqalgan muammolardan biri bo'lib, uni debug qilish jarayonida ortiqcha vaqt talab etadi. Chiqindi muammosi (obyektga hech qanday havola bo'lmaganda) kamdan-kam uchraydigan muammo bo'lsa-da, agar u juda noqulay paytda yuz bersa, oqibatlari achinarli bo'ladi. Ushbu muammolar Java'da yuzaga kelmaydi, chunki dasturchi aniq pointerlar bilan ishlamaydi. Xotirani boshqarish (chiqindilarni yig'ish) – bu dastur ijro muhitining masalasi bo'lib, dasturchi bu jarayonga aralashmaydi.

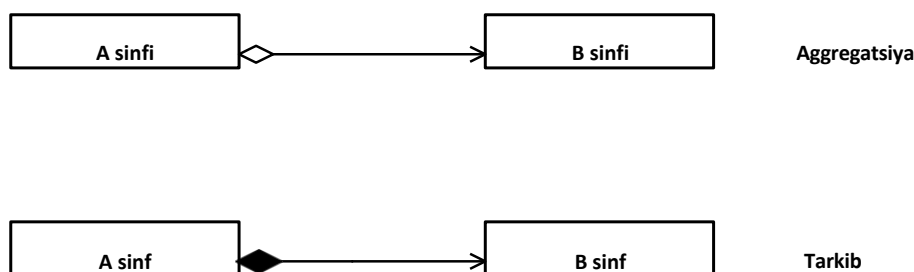
Ammo Java yuqorida tilga olingan muammolarni nafislik bilan hal etsa-da, loyihalash bosqichidagi bitta muammoni – agregatsiya muammosini – hal qilishni murakkablashtiradi. Loyihalash jarayonida qandaydir tarzda real ob'ektlar yoki jarayonlar modellashtiriladi. Har bir real ob'ekt tegishli xususiyatlarga va ma'lum turdagi ob'ektga tatbiq etiladigan tabiiy aniqlangan operatsiyalarga ega. Agar biror ob'ektning xususiyati o'z navbatida ob'ekt bo'lsa, dasturlash tillarida bunday modellashtirish (va bu ob'ektlar o'rtasidagi munosabat) agregatsiya deb ataladi. Matnda "obyekt" so'zi real dunyoning biror ob'ekt modelini anglatadi. Shunday qilib, birlashtiruvchi obyekt butun deb, obyekt-xususiyat esa qism deb ataladi. Masalan, hujjat o'z mazmunini birlashtiradi, kompyuter protsessorni, avtomobil esa dvigatelni birlashtiradi va hokazo.

Garchi agregatsiyaga kiritilgan obyekt qism obyekt sifatida tushunilsa-da, ba'zi bunday obyekt-qismlar jismonan kirituvchi obyekt ichida joylashmasligi mumkin. Masalan, kitob muallifi kitobdan tashqarida, shaxs bank hisobiga ega, uni jismonan o'zida jamlamaydi, aksiyador aksiyalarni to'liq egallaydi, lekin ular uning tarkibiga jismonan kirmaydi, avtomobil ro'yxatdan o'tkazish guvohnomasi avtomobildan tashqarida bo'lishi mumkin va hokazo. Bu munosabatlar, shubhasiz, agregatsiya munosabatlari bo'lib, ularning mohiyati jismoniy emas, balki konseptualdir. O'z ichiga oluvchi ob'ekt hanuzgacha butunlikni anglatadi, lekin uning qismi butun ichida bilvosita mavjud bo'ladi.

Ko'rsatkichlarga ega tillarda [1] butun va uning bilvosita qismi (xususiyat) munosabati ko'rsatkichlar orqali amalga oshiriladi, bu munosabatning obyekt-qismi esa qism-ko'rsatkich yoki qism-ulov deb ataladi. Butun ichiga jismonan kiritilgan xususiyatlar butun qismi uchun xotirani to'g'ridan-to'g'ri ajratish (kengaytirish) orqali amalga oshiriladi; bu munosabatda obyekt-qism qism-qiymat yoki kengaytirilgan qism deb ataladi. Shu tariqa, bu tillarda butun obyektning jismoniy chegarasi aniqlanadi. Obyekt ichida (chegaralar ichida) bo'lib, unga jismonan kiradigan hamma narsa

- uning qiymatlari tarkibiga kiritilgan (joylashtirilgan), va obyekt bilan agregatsiya aloqasiga ega bo'lgan, lekin jismonan uning ichida joylashmagan hamma narsa – obyektning (chegarasi) ichida tegishli qiymatlarga ko'rsatkichlar sifatida kiritilgan. [3]-da qiymatlari boshqa obyekt ichida joylashtirilgan obyekt turlari kengaytirilgan turlar deb ataladi.

UML [2] obyektga yo'naltirilgan dizayn tilida sinf diagrammasida bunday munosabatlarni modellashtirish uchun ikkita assotsiatsiya munosabati mavjud. Agregatsiya munosabati qism bilan butun o'rtasidagi bog'lanishni (butunning chegaralariga kirmaydigan) aniqlash uchun ishlatiladi. Kompozitsiya munosabati esa butun va uning kengaytirilgan qismi o'rtasidagi munosabatni ko'rsatish uchun qo'llanadi (15.1-rasmga qarang) [2].



15.1-rasm. Havola bo'yicha agregatsiya (yuqori) va qiymat bo'yicha agregatsiya (pastki)

Manba: [2]

Aslida, ko'rsatkich o'rniga, ushbu sinfning ma'lumotlar bazasi obyektlarida ushbu obyektning yozuv kalitidan foydalanishingiz mumkin. Bu ham agregatsiya, lekin jismoniy kiritishsiz.

Obyektning jismoniy chegaralari tushunchasi obyekt ustidagi ba'zi asosiy operatsiyalarning bajarilish semantikasi aniqlash uchun zarur. Bu operatsiyalar: yaratish, tayinlash, nusxalash (klonlash), solishtirish va o'chirish. Masalan, agar ob'ektni nusxalaysiz, qismlar – qiymatlar bo'lsa, ularni ushbu ob'ekt-qismlarga nusxalash, qismlar esa ko'rsatkichlar bo'lsa, ob'ekt-qismlarga ko'rsatkichlarni, ya'ni qismlarning manzillarini nusxalash kerak bo'ladi. Bundan tashqari, jismoniy chegarani aniqlash ob'ekt qismi bir nechta ob'ektlar o'rtasida baham ko'riladimi yoki u faqat uning egasi – butun ob'ektning xususiyatimi, deb aniqlash uchun zarur. Qismlar bir nechta sub'ektlar – masalan, egalar – o'rtasida taqsimlangan ko'plab misollar mavjud. Boshqa kitob ushbu muallifga ishora qilishi mumkin, ya'ni ikki kitobda bitta muallif bo'ladi va shuning uchun har ikkala kitob ham bir xil muallifga murojaat qiladi. Bank hisobvarag'i kitobi ma'lum bir mijoz hisobiga ishora qilishi, yo'l harakati politsiyasidagi ma'lumotlar bazasi esa ushbu ro'yxatdan o'tkazish guvohnomasiga ishora qilishi mumkin va hokazo.

Agar obyekt qiymati o'zgarsa, bu o'zgarish ushbu obyektning o'z qismlari sifatida ulashadigan va ko'rsatkich orqali unga murojaat qiladigan barcha obyektlarda aks ettiriladi. Haqiqiy hayotda ham xuddi shunday bo'lishi kerak. Darhaqiqat, agar kitob muallifining shaxsiy ma'lumotlarini o'zgartirsak, unda ushbu muallifning barcha kitoblari bo'yicha ma'lumotlar ham o'zgaradi. Xuddi shunday, agar bank hisobvarag'idagi balans yoki holatni o'zgartirsak, bu mijoz va bankning hisob kitob daftarlarida aks ettiriladi. Agar transport politsiyasi ma'lumotlariga asoslanib (masalan, bo'yagandan so'ng) avtomobil ro'yxatga olish guvohnomasi ma'lumotlari tuzatilsa, bu o'zgarish avtomobilning ro'yxatga olish guvohnomasida aks ettiriladi.

Kontseptual jihatdan, yuqoridagi operatsiyalar (yaratish, tayinlash va hokazo) obyektlarga nisbatan ikki testga asoslanadi: identiklikni tekshirish va ekvivalentlikni tekshirish. Identiklikni tekshirish – obyektlar real dunyoda bir xil obyekt ekanligini aniqlashdir. Bir xil sinfga mansub obyektlar har doim o'z mohiyati bo'yicha bir-biridan farq qiladi. Ekvivalentlikni tekshirish – obyektlarning holati bir xil ekanligini aniqlashdir [3].

Shu sababli, ko'rsatkichlarga ega tillarda obyektlar ustidagi yuqorida tilga olingan amallar tabiiy ravishda, ushbu ob'ektning jismoniy chegaralari doirasida standart amallar orqali aniqlanadi. Qism-ko'rsatkichlar bilan ishlaganda butun ob'ekt va qism ob'ekt bir-biridan alohida mavjud ekanligini yodda tutish kerak; tegishli sinf instanttsiyalarini mustaqil ravishda yaratish va yo'q qilish mumkin, qism-qiymatlar esa butun bilan birga "tug'iladi", uning ichida "yashaydi", u (butun) unga kirish huquqiga ega bo'lgan yagona ob'ekt va butun bilan birga "o'ladi".

Ushbu tillarda obyekt yaratilganda u o'zining jismoniy chegaralari doirasida yaratiladi. Qism-qiymatlar uchun tegishli sinf ob'ekti uchun joy ajratiladi, qism-ko'rsatkichlar uchun esa bu qism qaysi sinfga tegishli ekanligidan qat'i nazar, faqat ko'rsatkich uchun joy ajratiladi. Taqqoslashda obyektlar ham o'zlarining jismoniy chegaralari doirasida solishtiriladi: parts-values obyektlar sifatida taqqoslanadi, parts-pointers esa manzillar (ko'rsatkich qiymatlari) bo'yicha taqqoslanadi. Tayinlash operatsiyasi ham jismoniy chegaralar doirasida amalga oshiriladi: parts-pointers ko'rsatkich qiymati sifatida nusxalanadi, ya'ni manzil nusxalanadi, parts-values uchun esa obyekt nusxalanadi. Nusxalash operatsiyasi ham tayinlash operatsiyasi kabi amalga oshiriladi.

Ko'rsatkichlarga ega tillarda, yuqorida tilga olingan chegaralarni belgilash bo'yicha foyda va agregatsiyani loyihalashning tabiiy yondashuvi bilan birga, hal qilinishi kerak bo'lgan muammolar ham mavjud. Qoldiq referensiyalar va axlat muammolaridan tashqari, quyidagi jihatlarga e'tibor berish zarur: referensiyalar orqali strukturaviy bog'liqlikni oldini olish uchun obyektlarni yaratish va yo'q qilish bo'yicha har qanday tartibga rioya qilish muhim, chunki ularga ishora qiluvchi referensiyalar turli joylarda joylashishi mumkin. Kimdir buni bajarishi shart. Bundan tashqari, boshqa obyektдан boshqa obyektning bir qismi bo'lgan obyektga havola qilish (havola orqali agregatsiyani o'rnatish) imkonsizdir. Klassni loyihalashda klass turidagi obyektlarni qiymat bo'yicha to'g'ridan-to'g'ri yoki bilvosita rekursiv agregatsiya qilish taqiqlanadi.

Java tilida yuqorida keltirilgan muammolar o'z-o'zidan yo'qoladi. Ammo quyidagi savol tug'iladi, bu dizaynda aniq modellashtirishni ta'minlash nuqtai nazaridan juda muhim: Java'da ob'ekt chegarasini qanday aniqlash? Aslida javob – hech qanday. Barcha primitiv turdagi xususiyatlar shu turdagi maydon (field) sifatida, primitiv bo'lmaganlari esa ko'rsatkich (pointer) sifatida amalga oshiriladi.

Java tilida agregatsiya munosabati (qism-ko'rsatkichlarni modellashtirish) muammosiz amalga oshiriladi, chunki bu munosabat ko'rsatkichga asoslangan. Muammo faqat kompozitsiya munosabatiga, aniqrog'i – modellashtirilgan obyektning jismoniy chegarasini aniqlashga tegishli. Keyin aynan shu munosabat ko'rib chiqiladi, ya'ni qism-qiymatni amalga oshirish obyekt-qismni butunlikda kengaytirish orqali emas, balki boshqa usulda ko'rib chiqiladi.

Aslida, Java dasturining ijro etilishi paytida xotira tuzilmasiga nazar tashlasak (snapshot olsa), obyekt va uning qismi orqali bu qism-qiymatmi yoki qism-ko'rsatkichmi ekanligini ajratib bo'lmaydi, chunki barcha qismlar butun obyekt bilan ko'rsatkichlar orqali bog'langan.

Filozofik nuqtai nazar [8] mavjud bo'lib, unda munosabat birlamchi, ob'yekt esa ikkilamchi hisoblanadi. Haqiqiy dunyoda ob'yekt bu munosabatning cheksiz miqdordagi komponentlari o'rtasidagi munosabat sifatida namoyon bo'lishi mumkin. Dizayn jarayonida real dunyo modellari va abstrakt modellardan foydalaniladi. Shu nuqtai nazardan qaraganda, masalan, tekislikdagi nuqta ikki real son munosabati, kitob esa nom, nashriyot, muallif va mazmun o'rtasidagi munosabat, shaxs – ism, tug'ilgan sana, yashash manzili va ijtimoiy identifikator munosabati, avtomobil esa korpus, dvigatel, elektr tizimi, yurish qismi va

ro'yxatga olish guvohnomasi. Barcha munosabatli ma'lumotlar bazalari nazariyasi (munosabatli ma'lumotlar modeli) [7] aynan shunga asoslangan.

Shunday qilib, bu falsafiy nuqtai nazardan qaraganda, jismoniy chegaralar butun ob'ektning qattiqqligini (qismlarning ajralmasligini) emas, balki ushbu qismlarning zich bog'lanishini, ya'ni ob'ekt qismlari o'zaro va butun ob'ektni qamrab olishda yaqin o'zaro ta'sirda bo'lishini belgilaydi. Bu ta'sirlar turli ob'ektlar qismlari o'rtasidagi ta'sirlardan ko'ra kuchliroqdir. Ushbu munosabatlarni qo'llab-quvvatlovchi o'zaro ta'sir interfeysi doimiydir. Shunday qilib, butun ob'ektning modulli tuzilishi haqida gapirish mumkin. Har bir modul bo'lagi o'z interfeysi va mas'uliyatiga ega. Agar almashtirilayotgan bo'lak ushbu interfeysni qo'llab-quvvatlasa, butun obyekt tarkibidagi bo'lakni boshqa bo'lak bilan almashtirish mumkin.

15.2. KENGAYTIRILGAN OBYEKTAR

Ko'rsatkichlarga ega tillarda kengaytirilgan obyektlar tushunchasi mavjud. Ushbu tushuncha murakkab obyektlarni modellashtirish uchun ishlatiladi, ularning qismlari jismoniy chegaralar ichida "joylashtirilgan". To'liq bo'lmagan modullikni, ya'ni dinamik almashtinuvchanlikni to'liq qo'llab-quvvatlamaydigan joylashtirilgan obyektlar. Ichki joylashtirilgan qismlar bilan "tug'ilgan" murakkab ob'ekt ijro vaqtida shu interfeysni amalga oshiradigan boshqa ob'ekt bilan ichki qismini almashtira olmaydi. C++ misolida (15.2-rasm) polimorfizm buzilgan deb aytish mumkin.

```
#include <iostream>
using std::cout; using
namespace std;

class A {
private:
    int x;
public:
    A() {}
    int getX() {return x;}
    void setX(int i) {x=i;}
};

class B : public A {
private:
    int y;
public:
    B() { }
    B(int i) { y = i;}
```

```

        int getY() {return y;}
        void setY(int i) {y=i;}
    };
    class C {
    private:
        A a;
        int i; // ba'zi maydon
    public:
        C() { }
        A* getA() {return &a;};
        void setA(A a1) {a = a1;}
    };
    int main() {
        C c = C();
        B b1;
        b1.setX(10);
        b1.setY(12);
        cout << "Polimorfik tayinlashdan oldin obyekt maydonlarining qiymatlari:" <<
        endl; cout << "b1.x qiymati: " << b1.getX() << endl;
        cout << "b1.y qiymati: " << b1.getY() << endl;
        // polimorfik tayinlash
        c.setA(b1);
        // ko'rsatkichni e'lon qilish va A* dan B* ga turini o'zgartirish
        A * a = c.getA();
        B * b2 = (B*)a;
        cout << "Polimorfik tayinlashdan keyin obyekt maydonlarining qiymatlari:" <<
        endl; cout << "b2.x qiymati: " << b2->getX() << endl;
        cout << "b2.y qiymati: " << b2->getY() << endl; while
        (true) {};
        return 0;
    }

```

15.2-rasm. Misol 1

Manba: o'z ishi

15.2-rasmdagi dastur quyidagi natijani beradi:

```

Polimorfli tayinlashdan oldin obyekt maydonining qiymatlari:
b1.x qiymati:      10
b1.y qiymati:      12
Ob'ekt maydonining qiymatlari polimorfik tayinlashdan keyin:
b2.x qiymati:      10
b2.y qiymati:      65535

```

Bu yerda uchta sinf e'lon qilinadi: **A** sinfi, uning **B** subklassasi va kengaytirilgan qism-qiymatga ega **C** sinfi (**a** nomli). Asosiy funksiya tanasida **C** sinfidan (**c** nomli) obyekt yaratiladi. Yaratish jarayonida ushbu **C** sinfi obyekti ichida **A** sinfidan (**a** nomli maydoni) obyekt kengaytiriladi. Keyin **B** sinf ob'ekti (**b1** nomi bilan) yaratiladi va **uning** tegishli maydonlariga **10** hamda **12** qiymatlari beriladi. Ushbu uchta qiymat konsolga chiqariladi. Shuningdek, **c.setA(b1)** orqali

c obyektining a (**A** turidagi) maydoniga polimorf tarzda **B** sinfi obyektini (**b1** nomi bilan) tayinlanadi. **A* a = c.getA()** orqali a ko'rsatkichi **c.a** maydoniga ishora qiluvchi ko'rsatkich sifatida yaratiladi. Shunday qilib, a ko'rsatkichi aslida **B** sinfi obyektiga ishora qilgani uchun, uni **B** sinfi obyektini (**b2** nomi bilan) ko'rsatkichiga aylantirish (cast qilish) mumkin. Ushbu obyekt maydonlarining qiymatlari **b1** ning mos maydonlari qiymatlari bilan bir xil bo'lishi kerak, lekin dastur chiqishidan ko'rinib turibdiki, y maydonining qiymati boshqacha (12 emas, balki 65535).

Bu holat ajablanarli emas, chunki c ob'ekti yaratilganda uning ostida kengaytirilgan a ob'ekti hisobga olingan holda xotira ajratiladi. Keyin **b1** ob'ektini a ob'ektiga polimorfik tayinlash paytida y maydonining qiymati a ob'ekti uchun ajratilgan joyga sig'maydi. Chiqishdagi 65535 qiymati C++ dagi oddiy hiyla (zaif tip nazorati) natijasidir.

Shunday qilib, kengaytirilgan obyektlar bilan ishlaganda polimorfizm ishlamaydi, lekin modullik, qayta ishlatish va umuman obyektga yo'naltirilgan dasturlash polimorfizmi talab qiladi.

Aslida, zamonaviy agregat qurilmalar modullik tamoyillari asosida yaratilgan. Masalan, avtomobil o'zining jismoniy chegaralari ichida dvigatelni o'z ichiga olsa-da, uning dvigatelini boshqasiga, masalan, qo'shimcha dvigatel komponentlariga ega bo'lgan yanada zamonaviy dvigatelga almashtirish mumkin. Muhimi shundaki, yangi dvigatel eski dvigatelning joyiga sig'ishi va avtomobilning qolgan qismlari bilan barcha interfeys elementlarini yoqish imkoni bo'lishi kerak. Kompyuterda RAM yoki disk xotirasini, protsessorni almashtirish – bularning barchasi qo'llaniladi. Bugun inson uchun sun'iy a'zolar haqida ham shunday deyish mumkin. Yuqoridagi misollar shuni ko'rsatadiki, jismoniy chegaralar monolitiklik yoki butunlikni anglatmaydi; aksincha, u asbob-uskunalar ichidagi elementlar o'rtasidagi aloqalar yaqinroq, ichki jismoniy chegaradagi elementlar bilan tashqi dunyo o'rtasidagi aloqalar esa zaif bo'lishini anglatadi.

Fizik chegaralar qism ham obyekt ekanligini ta'kidlaydi va printsip jihatidan uni mustaqil deb hisoblash mumkin. Ba'zi komponentlarning qamrab oluvchi obyektga a'zoliq statik ravishda hech qayerda ko'rsatilmagan. Bu yuqorida tilga olingan yaratish, tayinlash, nusxalash (klonlash), solishtirish va o'chirish operatsiyalarining semantikasi bilan belgilanadi.

Yuqorida ko'rilganidek, ko'rsatkichlarga ega tillarda jismoniy chegaralarni (kompozitsiyani) to'g'ri modellash tirish kengaytirilgan qismlar yordamida amalga oshirib bo'lmaydi. Java tilida esa hammasi o'ylab topilgan, chunki kompozitsiya ko'rsatkichlar asosida amalga oshiriladi.

Java tili bo'yicha ko'plab kitoblar [4, 5] ushbu dizayn muammosidan qochadi. Dizaynga bag'ishlangan [6] kabi kitoblarda esa kompozitsiya va agregatsiyani amalga oshirish usullari ochilmagan. Michelle Yaiser [9] faqat yaratish operatsiyasiga asoslangan kompozitsiya va oddiy agregatsiyani muhokama qiladi.

15.3. AGGREGATSIYA

Agregatsiya va kompozitsiya ko'rsatkichlar yordamida amalga oshirilishi kerak. Ko'rsatkich-qism amalga oshirilganda (oddiy agregatsiya), u alohida yaratilib, butun ob'ekt yaratilganda konstruktorgacha parametr sifatida (yoki boshqa usulda) uzatiladi – 15.3-rasmdagi 2-misolga qarang.

```
class A { private
    int x; public
    A() {}
    public A(int x) {this.x = x;}
    public int getX() {return x;}
    public void setX(int x) {this.x = x;}
};
class B extends A {
    private int y;
    public B() { }
    public B(int y) {this.y = y;}
    public B(int x, int y) { super(x); this.y = y;} public
    int getY() {return y;}
    public void setY(int y) {this.y = y;}
}

class C {
    private A a;
    private int c; // ba'zi maydon
    public C(A a) {this.a = a;}
    public A getA() {return a;};
    public void setA(A a) {this.a = a;}

    public static void main(String[] args) {
        B b1 = new B(10,12);
        C c = new C(b1);
        System.out.println("Obyekt maydonining qiymatlari o'zgartirishdan oldin:");
        System.out.println("x qiymati: " + ((B)c.getA()).getX());
    }
}
```

```

        System.out.println("y qiymati: " + ((B)c.getA()).getY());
        System.out.println("o'zgarishdan oldin ob'ekt maydon qiymatlari:");
        b1.setX(24);
        b1.setY(48);
        System.out.println("Obyekt maydonining qiymatlari o'zgartirilgandan keyin:");
        System.out.println("x qiymati: " + ((B)c.getA()).getX());
        System.out.println("y qiymati: " + ((B)c.getA()).getY());
    }
}

```

15.3-rasm. Misol 2

Manba: o'z ishi

15.3-rasmdagi dastur quyidagi natijani beradi:

```

Obyekt maydonining qiymatlari o'zgartirishdan oldin:
b1.x qiymati: 10 b1.y
qiymati: 12
Obyekt maydonining qiymatlari o'zgartirishdan oldin:
Obyekt maydonining qiymatlari o'zgartirilgandan keyin:
b1.x qiymati: 24 b1.y
qiymati: 48

```

```

Obyekt maydonining qiymatlari o'zgartirishdan oldin:
b1.x qiymati: 10 b1.y
qiymati: 12
Obyekt maydonining qiymatlari o'zgartirishdan oldin:
Obyekt maydonining qiymatlari o'zgartirilgandan keyin:
b1.x qiymati: 24 b1.y
qiymati: 48

```

Bu yerda **c** sinfi kodiga qarasangiz, **c** sinf ob'ektini yaratishda konstruktorga **a** sinf ob'ektini uzatish zarurligini ko'rishingiz mumkin. **c** sinf ob'ekti o'zining **a** nomli maydoni bilan agregatsiya munosabatiga (qism-ko'rsatkich) kiritilgan. Asosiy koddan ko'rinib turibdiki, **a** sinfi ob'ekti o'rniga polimorfik tarzda **b** sinfi (**b1** nomli) ob'ektini uzatish mumkin, chunki **b** sinfi **a** sinfining subklassidir. Agar **b1** maydonlarining qiymatlarini o'zgartirsak, **c.getA()** orqali kirish mumkin bo'lgan maydonlarning qiymatlari ham o'zgaradi.

15.4. KOMPOZITSIYA

Yuqorida muhokama qilinganidek, kompozitsiyada yaratish, tayinlash, nusxalash (klonlash), solishtirish va o'chirish operatsiyalarining semantikasi aniqlash uchun jismoniy chegarani amalga oshirishimiz kerak. Ushbu operatsiyalarni batafsil ko'rib chiqamiz.

Kompozitsiyani qurish (yaratish)

Kompozitsiya nuqtai nazaridan obyekt o'zining fizik chegarasi bilan "tug'iladi", ya'ni uning barcha qismlari ularni o'rab turuvchi obyekt bilan birga yaratiladi. Butun qismlari tashqaridan boshqa bog'lanishlarga ega bo'lmashligi kerak. Kompozit obyektни konstruktorgacha barcha qismlarini parametr sifatida uzatish orqali yaratish mumkin. p1 P1 turidagi, p2 P2 turidagi, ..., pn PN turidagi bo'lsin - bu C sinfi ob'ektining alohida qismlari bo'lib, bu turdagi ob'ektlarda boshqa qismlar yo'q. Unda 15.4-rasmdagi shablon C sinfi konstruktori uchun ishlatilishi mumkin.

```
class C { private
    P1 p1; private
    P2 p2;
    ... private
    PN pn;
    public C(P1 p1, P2 p2,..., PN pn) { // Qismlarni dastlabki holatga
        keltiruvchi konstruktor this.p1 = p1.clone();
        this.p2 = p2.clone();
    ...
        this.pN = pN.clone();
    }
    public C(...,P1 p1, P2 p2,..., PN pn, ...) { // Boshqa konstruktorlar C(p1,p2,...,pn);
        ...
    }
}
```

15.4-rasm. Misol 3

Manba: o'z ishi

Barcha konstruktorlar qismlarni dastlabki holatga keltiradigan sinf konstruktorini chaqirishi kerak. Boshqa obyektlarga havolalar paydo bo'lishining oldini olish uchun o'tkazilgan parametr obyektlariga havolalar o'rnatilmaydi, buning o'rniga ushbu obyektlarning klonlariga havola o'rnatiladi. Ushbu konstruktor obyekt qismlarining klonlarini obyekt maydonlariga tayinlaydi. p1, ..., pn obyektlarning klonlash siyosati mos ravishda P1, ..., PN sinflarini aniqlaydi.

Taqqoslash

Tayinlash amallari, taqqoslash va nusxalash (klonlash) ikki ob'ekt ishtirokidagi ikki argumentli amallardir.

Taqqoslash operatsiyasining natijasi `boolean` turiga ega. Shunday qilib, bu operatsiya ikki ob'ekt o'rtasidagi ekvivalentlik munosabatidir.

Ma'lumotlarni solishtirish uchun dasturlash tillari odatda "==" (yoki Pascalga o'xshash tillarda "=") operatsiyasidan foydalanadi. Ko'rsatkichlarga ega tillarda bu operatsiya orqali qism-qiymatlarning barcha qiymatlari solishtiriladi, qism-ko'rsatkichlar bo'yicha esa ushbu ko'rsatkichlarning qiymatlari taqqoslanadi. Oldin aytib o'tilganidek, shu tarzda obyektning jismoniy chegarasi aniqlanadi. C++ tilida satr tushunchasi (uning jismoniy chegarasi bilan) qator mazmunini o'z ichiga oladi, shuning uchun C++ da satrlar ko'rsatkich sifatida amalga oshirilgani bois "==" operatsiyasi ustun qo'yilgan va satr mazmunini solishtiradi, C tilida esa bu maqsad uchun **strcmp** funksiyasidan foydalaniladi. Java tilida bu operatsiya primitiv ma'lumot ob'ektlarini solishtirish uchun ishlatiladi. Ob'ektlar uchun bu operatsiyaning qo'llanilishi ularning qiymatidan emas, balki manzillarini solishtirish semantikasi bilan amalga oshiriladi. Simvolli satrlarni (**String** turidagi ob'ektlarni) solishtirish uchun **equals**, **equalsIgnoreCase** va **compareTo** usullari ishlatiladi:

- **boolean equals(Object other)** usuli satrlarni ularning mazmunini aniq moslik bo'yicha solishtirish uchun ishlatiladi. Agar satrlar mos kelmasa, false qaytaradi. U Object sinfidan meros qilib olingan usulni ustun qo'yadi. Ushbu usul keyinchalik batafsil muhokama qilinadi.
- **boolean equalsIgnoreCase(String other)** – joriy satr va parametr sifatida berilgan satr katta-kichiklikni hisobga olmasdan mos kelishi tekshiriladi.
- **int compareTo(String other)** - joriy satr va parametr sifatida berilgan satr leksikografik tartib bo'yicha tekshiriladi. Agar satr kamroq bo'lsa, -1 qaytaradi; agar katta bo'lsa +1, aks holda (agar ular teng bo'lsa) 0 qaytaradi.

Keyingi bosqichda Java'da obyektlarni solishtirishni kengroq ko'rib chiqamiz.

[4]-sonli kitobning 5.2-bo'limida Object sinfi equals metodi muhokama qilingan. Ushbu metod barcha sinflarga meros qilib o'tadi va joriy obyekt (masalan, A sinfini) boshqa bir obyekt bilan solishtirish zarur bo'lganda aynan shu metod qo'llanadi. Taqqoslash semantikasini (nima solishtirilishi, nimaga va qanday) meros qilib olingan metodni ustun qo'yish (override) orqali ushbu metodda aniqlanadi. [4] ga asoslangan test namunasi 15.5-rasmda ko'rsatilgan.

```

public boolean equals(Object o) {
    // obyektlarning bir xilligini aniqlash uchun tezkor
    test if (this == o) return true;
    // aniq parametr null bo'lsa, false qaytarishi kerak if (o
    == null) return false;
    // agar sinflar mos kelmasa, ular teng bo'la olmaydi
    if (getClass() != o.getClass()) return false;
    // endi otherObject nol bo'lmagan Employee A
    ekanligini bilamiz other = (A)o;
    // maydonlarning qiymatlari bir xilmi, deb tekshirish
    return // obyekt maydonlarini tekshirish;
}

```

15.5-rasm. Misol 4

Manba: [4]

Solishtirilishi kerak bo'lgan Object, ya'ni o deb nomlangan obyekt metodga Object turidagi parametr sifatida uzatiladi.

Odatda, ma'lum bir sinf uchun ushbu metod ustun qo'yilganda faqat maydonlarning taqqoslanishi boshqacha bo'ladi. Nima nima bilan va qanday taqqoslanishi shu yerda aniqlanadi. Faraz qilaylik, A sinfiga mansub biror (taqqoslanayotgan) obyekt o'zining jismoniy chegaralari ichida (tarkibiy munosabatda) B sinfiga mansub boshqa bir obyektning qism sifatida o'z ichiga oladi, masalan, f maydoni orqali havola qilingan. Keyin biz ushbu qismni taqqoslanadigan obyektning mos qismlari bilan quyidagicha solishtiramiz:

```

...
& getFieldF().equals(o.getFieldF())
...

```

bu yerda getFieldF() f nomidagi obyektga ishora qiluvchi ko'rsatkichni qaytaradi.

Ob'ektning jismoniy chegaralariga kiritilmagan qismlarni solishtirishga ruxsat berilmaydi. Odatda ikki kompyuterni solishtirayotganda ularning egasini, qayerda sotib olinganini va hokazolarni solishtirish shart emas. Ikki avtomobilni solishtirayotganda, agar bu ma'lumotlar avtomobilning jismoniy chegaralariga kiritilmagan bo'lsa, odatda ularning texnik pasportlarini, ro'yxatga olish manzilini va hokazolarni solishtirishga hojat yo'q. Solishtirish ob'ektning jismoniy chegaralari aniq belgilangan moddiy modelga asoslanib amalga oshirilishi kerak.

Bundan tashqari, taqqoslash matematik ma'nodagi ekvivalentlik munosabatlarning xususiyatlariga asoslanadi:

- refleksivlik xususiyati: `x.equals(x) == true;`
- simmetriya xususiyati: `x.equals(y) <=> y.equals(x);`
- transitiv xususiyat: `x.equals(y) & y.equals(z) => x.equals(z);`

- moslik xususiyati: turli chaqiruvlarda `x.equals(y)` chaqiruvlari, agar tenglikda ishlatilgan qiymatlarda hech qanday o'zgarish bo'lmasa, bir xil natijani qaytaradi;
- bo'sh emaslik xususiyati: `x.equals(null) == false`.

[4] da aytilishicha, sinflar ierarxiyasida bir sinf obyektini uning subclass obyekti bilan solishtirish zarur bo'lganda, simmetriya xususiyatini buzmaslik uchun subclass obyektini solishtirish semantikasini aniqlash kerak. Agar semantika o'zgarmasa, ya'ni subclass obyektlarining ekvivalentligi superklassdagidek bo'lsa, unda superklassning equals metodi instanceof predikatidan foydalanishi va final kalit so'ziga ega bo'lishi kerak. Agar semantika o'zgarsa, subclass equals metodini ustun qo'yadi (override qiladi), va superklassda sinflarning identikligini tekshirish uchun `getClass()` metodu ishlatiladi.

Yuqoridagi taqqoslash usuli ob'ekt ekvivalentligi deb atalishi mumkin, chunki ob'ekt o'zini boshqa ob'ekt bilan o'z sinfining ekvivalentlik semantikasiga asoslanib solishtiradi. Ammo biz abstrakt (sinf) ekvivalentlik haqida ham gapirishimiz mumkin, bu yerda ikki ob'ekt qandaydir abstrakt nuqtai nazardan solishtiriladi. Masalan, qalamni va yog'och tayoqni ularning uzunligi va qalinligi bo'yicha, avtomobilni va ferris g'ildiragidagi kapsulani o'rindiqlar soni bo'yicha solishtirish mumkin va hokazo. Ushbu taqqoslash uchun shu abstraksiyaga mos keladigan sinf yaratiladi va bu sinfda equals metodi amalga oshiriladi. Ushbu metod obyektlarni shu sinfning ekvivalentlik semantikasiga ko'ra taqqoslaydi. Bundan tashqari, agar taqqoslash uchun qiymatlarga kirish bo'yicha maxsus metodlar talab qilinsa, taqqoslash uchun ushbu kirish metodlarini e'lon qiluvchi interfeys yaratiladi. Ushbu interfeysni amalga oshiradigan istalgan sinf obyektlari shu sinf nuqtai nazaridan taqqoslanishi mumkin.

15.6-rasmdagi 5-misol obyektli va abstrakt taqqoslashni ko'rsatadi.

```
interface ExistX {
    int getX();
}

class EqualByX {
    public static boolean equals (ExistX o1, ExistX o2) {
        return o1.getX() == o2.getX();
    }
}
```

```

class SuperClass implements ExistX {
    private int x;
    public SuperClass() {}
    public SuperClass(int x) {this.x = x;}
    public int getX() {return x;}
    public void setX(int x) {this.x = x;} public
    boolean equals(Object otherObject) {
        // obyektlar bir xil ekanligini tekshirish uchun tezkor
        test if (this == otherObject) return true;
        // aniq parametr null bo'lsa, false qaytarishi kerak if
        (otherObject == null) return false;
        // sinflar mos kelmasa, ular teng bo'la olmaydi
        if (!(otherObject instanceof SuperClass)) return false;
        // endi biz otherObject nol bo'lmagan SuperClass ob'ekti ekanligini
        bilamiz SuperClass other = (SuperClass) otherObject;
        // maydonlarning qiymatlari bir xilmi, tekshirish
        return x == other.x;
    }
    public static boolean equals (Object o1, Object o2) {
        if (!(o1 instanceof SuperClass)&&o2 instanceof SuperClass)) return false;
        SuperClass s1 = (SuperClass)o1;
        SuperClass s2 = (SuperClass)o2;
        return s1.getX() == s2.getX();
    }
};

class SubClass extends SuperClass { private
    int y;
    public SubClass() {}
    public SubClass(int y) {this.y = y;}
    public SubClass(int x, int y) { super(x); this.y = y;}
    public int getY() {return y;}
    public void setY(int y) {this.y = y;} public
    boolean equals(Object otherObject) {
        if (!super.equals(otherObject)) return false;
        // super.equals bu obyekt va otherObject bir xil sinfga mansubligini tekshiradi
        if (!(otherObject instanceof SubClass)) return false;
        // if (getClass() != otherObject.getClass()) return false;
        SubClass other = (SubClass) otherObject;
        return this.y == other.y;
    }
}

class Equals implements ExistX { private
    SuperClass a;
    private SubClass b;
    private int x;
    public Equals(int a, int b, int x) {
        this.a = new SuperClass(a);
        this.b = new SubClass(a, b);
        this.x = x;
    }

    public SuperClass getA(){return a;}
    public SubClass getB(){return b;} public
    int getX() {return x;}
    public static void main(String[] args) {

```

```

Equals e = new Equals(10, 12, 10);
System.out.println("---- Ob'ektlardagi a va b tengligi degani -----");
System.out.println("Object a is equal with object b:" + e.getA().equals(e.getB()));
System.out.println("Object b is equal with object a:" + e.getB().equals(e.getA()));
System.out.println("---- Ob'ektda a va b tengligi degani -----");
System.out.println("ExualByX bo'yicha a obyekt b obyekt bilan teng degani:" +
    EqualByX.equals(e.getA(), e.getB()));
System.out.println("ExualByX bo'yicha obyekt b obyekt a bilan teng: " +
    EqualByX.equals(e.getB(), e.getA()));
System.out.println("---- ExualByX bo'yicha a va e tengligi degani -----");
System.out.println("Object a ExualByX bo'yicha e obyekt bilan teng degani:" +
    EqualByX.equals(e.getA(), e));
System.out.println("ExualByX bo'yicha e obyekt a obyekt bilan teng degani:" +
    EqualByX.equals(e, e.getA()));
System.out.println("---- statik equals metodi bo'yicha a va e tengligi ----");
");
System.out.println("Object a is equal with object e in ExualByX means:" +
    SuperClass.equals(e.getA(), e.getB()));
System.out.println("Object e is equal with object a in ExualByX means:" +
    SuperClass.equals(e.getB(), e.getA()));
}
}

```

15.6-rasm. 5-misol

Manba: [4]

15.6-rasmdagi dastur quyidagi natijani beradi:

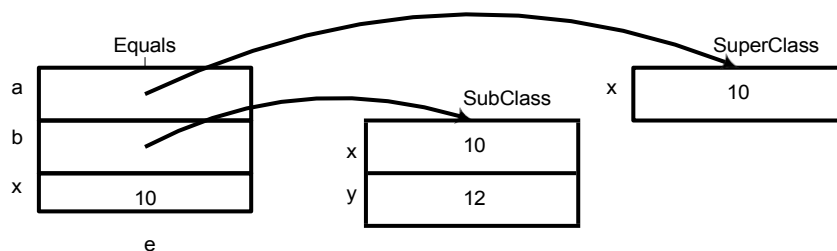
```

---- Ob'ektlardagi a va b tengligi ----Ob'ekt a
ob'ekt b ga teng:true
Ob'ekt b ob'ekt a bilan teng emas:false
---- Ob'ektdagi a va b tengligi ----Ob'ekt a ExualByX
bo'yicha ob'ekt b ga teng:true Ob'ekt b ExualByX bo'yicha
ob'ekt a ga teng:true
---- a va e ning ExualByX bo'yicha tengligi ----Ob'ekt a
va ob'ekt e ExualByX bo'yicha teng:true Ob'ekt e va
ob'ekt a ExualByX bo'yicha teng:true
---- a va e ning static equals metodidagi tengligi ----Object
a ExualByX bo'yicha e ob'ekti bilan teng:true Object e
ExualByX bo'yicha a ob'ekti bilan teng:true

```

15.6-rasmdagi kodda **ExistX** nomli bitta interfeys va **EqualByX**, **SuperClass**, **SubClass** hamda **Equals** sinflari e'lon qilingan. **Equals** sinfining **main** metodida bajariladigan jarayonni ko'rib chiqaylik.

Equals sinfi ob'ekti e yaratiladi, unda **SuperClass** va **SubClass** sinflarining **a** va **b** nomli ob'ektlari agregatsiyalangan. Bundan tashqari, **x** maydoni va uning qiymati ham mavjud. Olingan ob'ekt sxemasi 15.7-rasmda ko'rsatilgan.



15.7-rasm. Ob'ektlar sxemasi, 15.6-rasmdagi dasturga asoslangan.

Manba: o'z ishi

`e.getA().equals(e.getB())` yordamida **a** maydoni ko'rsatayotgan obyekt **b** maydoni ko'rsatayotgan obyekt bilan solishtiriladi. Obyektlar **SuperClass** sinfi asosida solishtiriladi. Ushbu sinfda **Object** sinfining `equals` metodi ustun qo'yilgan (override qilingan). Bu holda solishtirish uchun o'zining **SubClass** ost sinfidagi obyekt parametr sifatida uzatiladi. Solishtirishda quyidagi algoritm qo'llaniladi:

1. Agar obyektlarning manzillari teng bo'lsa, obyektlar ekvivalent hisoblanadi
2. Agar solishtirilayotgan obyekt bo'sh bo'lsa, obyektlar teng emas
3. Agar solishtirilayotgan obyektning turi **SuperClass** ga polimorfik bo'lmasa, obyektlar teng emas
4. Aks holda, taqqoslanayotgan obyekt **SuperClass** sinfi obyekt deb qabul qilinadi va **x** maydonining qiymatlari (**SuperClass** sinfining taqqoslash semantikasi) solishtiriladi. Bu yerda sinflarning aniq mos kelishi ishlatilmaydi, bu esa subklass obyektlarini polimorfik taqqoslashga imkon beradi.

Ushbu taqqoslash 4-taqqoslash elementini tekshirgach ijobiy natija qaytaradi, ya'ni taqqoslanayotgan obyektlar teng deb topiladi. Bundan tashqari, `e.getB().equals(e.getA())` yordamida ko'rinishicha simmetrik taqqoslash amalga oshiriladi – **b** maydoni ko'rsatgan obyekt **a** maydoni ko'rsatgan obyekt bilan solishtiriladi. Obyektlar **SubClass** sinfi asosida taqqoslanadi, u **SuperClass** sinfidagi taqqoslash metodini qayta ta'riflagan va taqqoslash quyidagi algoritm bo'yicha bajariladi:

1. Agar ular **SuperClass** sinfidagi `equals` metodi bo'yicha teng bo'lmasa, obyektlar teng emas
2. Agar solishtirilayotgan obyektning turi **SubClass** ga polimorfik mos kelmasa, obyektlar teng emas. Bu yerda subclass obyektlarini polimorfik solishtirish imkonini berish uchun sinflarning aniq mos kelishi qo'llanilmaydi.
3. Aks holda solishtirilayotgan obyekt **SubClass** deb qabul qilinadi va qiymatlari **y** maydoni solishtiriladi (**SubClass** sinfi solishtirish semantikasiga muvofiq).

Dasturda ko'rsatilgan qiymatlarga asosan obyektlar teng emas, chunki parametr orqali uzatilgan **SuperClass** sinfi obyekt **y** nomli maydonga ega emas. Eslatib o'tish joizki, bu ikki munosabat bir xil emas: birinchisi **SuperClass** sinfi asosida ikki obyektning solishtiruvchanligi, ikkinchisi **SubClass** sinfi asosida ikki obyektning solishtiruvchanligi, shuning uchun bu yerda simmetriya bo'lmaydi: **SuperClassEquality** (**a**, **b**) $\neq$ **SubClassEquality** (**A**, **b**).

EqualByX.equals (**e.getA** (), **e.getB** ()) yordamida **a** va **b** maydonlari bo'yicha ko'rsatilgan obyektlar **EqualByX** sinfi nuqtai nazaridan abstrakt tarzda solishtiriladi. Ushbu sinf **ExistX** turidagi obyektlarni solishtira oladi. **SuperClass** sinfi ham, **SubClass** sinfi ham ushbu interfeysni amalga oshirgani uchun, ularning obyektlari mazkur sinfning solishtirish semantikasiga muvofiq solishtirilishi mumkin. Ikkala ob'ektda ham **x** maydoni mavjud va bu maydonning qiymati bir xil, shuning uchun ular shu nuqtai nazardan teng. Quyidagi taqqoslash - **EqualByX.equals** (**e.getB** (), **e.getA** ()) yuqoridagi taqqoslashga simmetrik bo'lib, ikkala taqqoslash ham **EqualByX** sinfi asosida bajariladi va shuning uchun ular bir xil natija beradi.

Quyidagi ikkita taqqoslash ham mavhum bo'lib, **EqualByX** asosida qurilgan, lekin bu yerda butun obyekt (**e**) uning qismi (**a**) bilan **ExistX** sinfining taqqoslash semantikasi yordamida solishtiriladi. Shu nuqtai nazardan, har ikkala (to'g'ridan-to'g'ri va simmetrik) taqqoslash ijobiy natija beradi.

Oxirgi taqqoslashlarda **a** va **b** maydonlari bilan ko'rsatilgan obyektlar **SuperClass** sinfi nuqtai nazaridan abstrakt tarzda taqqoslanadi. Ushbu sinf **Object**'dan meros qilib olingan va o'zida qayta ta'riflangan **equals** taqqoslashning standart usulidan tashqari, **Object** turidagi ikki obyektни uzatadigan statik **equals** taqqoslash usuliga ham ega. Taqqoslash algoritmi quyidagicha:

1. Agar solishtirilayotgan obyektlarning turlari polimorfik **SuperClass** bo'lmasa, obyektlar ekvivalent hisoblanmaydi
2. Aks holda, obyektlar **SuperClass** deb qabul qilinadi va **x** maydonining qiymatlari (**SuperClass** sinfining taqqoslash semantikasi) solishtiriladi. Bu yerda aniq sinf mosligini tekshirish ishlatilmaydi, bu esa subklass obyektlarini polimorfik taqqoslashga imkon beradi.

Ushbu taqqoslashlar (to'g'ridan-to'g'ri va simmetrik) ijobiy natija beradi.

Standart Java kutubxonasida - `java.util.Arrays`, `static boolean equals (type [] a, type [] b)` statik metodi primitiv turlardagi massivlarni solishtirish uchun amalga oshirilgan, ya'ni `int`, `long`, `short`, `char`, `byte`, `boolean`, `float`, `double` turlari).

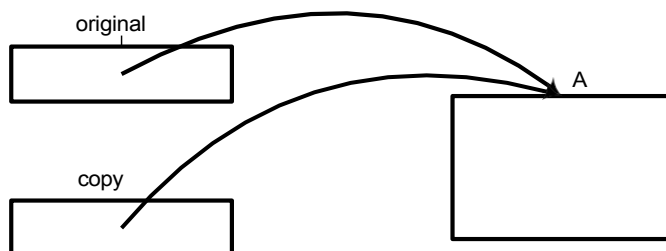
Tayinlash va nusxa olish (klonlash)

Obyektlarni tayinlash va nusxalash o'xshash amallardir. C++da obyektning barcha kengaytirilgan qismlari nusxalanadigan standart tayinlash mavjud, lekin siz bu amaliyotni ustun qo'yib, unga boshqa semantika berishingiz mumkin. Obyektlarni tayinlash semantikasiga va parametrlarni qiymat bo'yicha uzatish semantikasiga bir xil ma'no beriladi. Standart nusxalash amaliyoti (standart nusxalash konstruktori) va tayinlash bir xil ma'noni anglatadi. Shuningdek, standart nusxalash konstruktori ustun qo'yilishi mumkin.

Java'da ob'ektni tayinlashda faqat uning manzili nusxalanadi. Quyidagi parchada:

```
A original = new A();  
A copy = original;
```

original va **copy** o'zgaruvchilari bir xil obyektga ishora qiladi – 15.8-rasm.



15.8-rasm. Oddiy nusxa ko'chirishda o'zgaruvchining ko'rsatkichlari

Manba: o'z ishi

Fizik chegarani kesib o'tayotgan qismlarni nusxalash (klonlash) uchun `Object` sinfining himoyalangan (**protected**) `clone()` metodidan yoki serializatsiya (linearizatsiya) usulidan foydalanishingiz kerak. Keling, bu yondashuvlarni alohida ko'rib chiqaylik.

Standart JDK8 hujjatiga [10] ko'ra, clone metodi quyidagi imzo bilan berilgan:

```
protected Object clone() throws CloneNotSupportedException
```

Ma'lum bir sinf ob'ektini klonlash uchun ushbu sinf `Cloneable` interfeysini amalga oshirishi kerak. Ushbu interfeys sinfni hech qanday metodni amalga oshirishga majburlamaydi, faqat ko'rsatilgan sinfda ushbu interfeysning rasmiy amalga oshirilishi (bu tekshiruv `clone()` metodida bajariladi) muvaffaqiyatli bo'lganligini tekshirish kifoya. `x.clone()` chaqiruvi o'zgaruvchi `x` ko'rsatgan obyektning yangi nusxasini (klon) yaratadi, ammo bu klonlashning aniq ma'nosi `x` ko'rsatgan obyekt sinfiga bog'liq. Yuqoridagi hujjatga ko'ra, ushbu metodni override qilganingizda, qaytarilgan klon `super.clone()` orqali olinishi kerak.

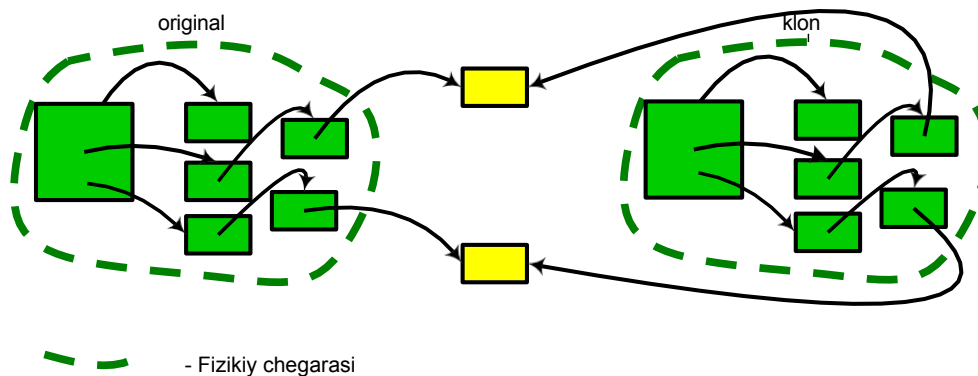
Keyin quyidagi ifoda bajariladi:

- `x.clone() != x.` // Bu `Object` sinfi tomonidan ta'minlanadi.
- `x.clone().getClass() == x.getClass()`
- `x.clone() != x.` // Bu `Object` sinfi tomonidan taqdim etilgan.
`x.clone().getClass() == x.getClass()`

Dokumentatsiyada [10] va Java kitoblarida, masalan [4], chuqur va yuzaki klonlash haqida gapiriladi. Aslida, obyektning yaratish jarayonida uni uning jismoniy chegaralari ichida klonlash zarur.

Ob'ektning jismoniy chegarasi ichida klonlangan ob'ektni manbadan mustaqil qilish uchun zarur bo'lsa, `super.clone()` chaqirilgandan so'ng olingan ob'ektda ba'zi "o'zgartirishlar" kiritish kerak bo'lishi mumkin. Agar obyektning subobyekti uning jismoniy chegarasiga kirmagan bo'lsa, hech narsa qilish shart emas; aks holda, ya'ni subobyekt jismoniy chegaraga kirsa, bu qismni nusxalab, ko'chirilgan obyektning ko'rsatkichini o'rnatish kerak (15.9-rasm).

15.9-rasmda asl "original" obyektida yashil rangdagi to'g'ri burchaklar ushbu obyektning jismoniy chegarasiga kiradigan qismlarni ifodalaydi. Bu qismlar nuqtali yashil chiziq bilan ko'rsatilgan. Sariq rang esa uning jismoniy qismiga kirmaydigan obyektlarni bildiradi. Klonlash natijasi "clone" deb ko'rsatilgan.



15.9-rasm. Murakkab obyektning klonlash

Manba: o'z ishi

6-misol (15.10-rasm) jismoniy chegaralar ichida klonlashni ko'rsatadi.

```
package Examples;
public class ForClone implements Cloneable{
    private int intField;
    public ForClone (int i) {
        intField = i;
    }
}
```

```

    }
    public ForClone clone() throws CloneNotSupportedException { ForClone
        cloned = (ForClone)super.clone();
        return cloned;
    }
    public void setInt (int i) {intField = i;}
    public int getInt() {return intField;}
}

package Examples;
public class Cloning implements Cloneable{ private
    double doubleField;
    private ForClone forField;
    public Cloning (int i, double d) {
        forField = new ForClone(i); doubleField
        = d;
    }
    public Cloning clone() throws CloneNotSupportedException {
        Cloning cloned = (Cloning)super.clone(); cloned.forField =
        forField.clone();
        return cloned;
    }
    public static void main (String[] args) throws CloneNotSupportedException {
        Cloning sc = new Cloning(12, 12.12);
        Cloning scNew = (Cloning)sc.clone();
        scNew.doubleField = 24.24;
        scNew.forField.setInt(24);
        System.out.println("Original = "+sc.doubleField);
        System.out.println("Clone = "+scNew.doubleField);
        System.out.println("Original = "+sc.forField.getInt());
        System.out.println("Clone = "+scNew.forField.getInt());
    }
}

```

15.10-rasm. Misol 6

Manba: [4]

main (main) metodi yaratiladi (sc) va u klonlanadi (scNew). Klonlashdan so'ng, klon maydonlari tuzatiladi: oddiy double (doubleField) tipi va ForClone tipi (forField) bilan. forField maydoni ushbu obyektning jismoniy chegarasiga kirgani uchun, u ham klonlanadi. Buning uchun ushbu obyektning clone metodi chaqiriladi va u klonlashni biladi.

Natijadan ko'rinib turibdiki, sozlash faqat nusxaga ta'sir qiladi va asl ob'ektning maydonlari o'z qiymatlarini o'zgartirmaydi:

```

Original = 12.12
Kloni = 24.24
Original = 12
Klone = 24

```

Massivlar bilan ishlashda ham `clone()` funksiyasini qo'llash mumkin. Bu holda massiv elementlari qiymatlari yangi yaratilgan massivga nusxalanadi. Shunday qilib, agar massiv elementlari primitiv turga mansub bo'lsa, unda aynan shu primitiv qiymat nusxalanadi; agar element obyekt (ya'ni ko'rsatkich) bo'lsa, unda ko'rsatkich nusxalanadi va natijada asl hamda nusxa elementlar bir xil obyektga ishora qiladi. Misol tariqasida, 1-rasmdan olingan parcha.

15.11 ga murojaat qilish mumkin: asosiy metodning davomi.

```
Cloning[] scArr = new Cloning[4];
scArr[0]= sc;
scArr[1]= sc;
scArr[2]= sc;
scArr[3]= sc;
for (Cloning c : scArr) {
    System.out.println("Manba: double = "+c.doubleField);
    System.out.println("Manba:      forField.int = "+c.forField.getInt());
}
Cloning[] scArrClone = scArr.clone();
scArrClone[1].doubleField = 36.36;
scArrClone[1].forField.setInt(36); for
(Cloning c : scArr) {
    System.out.println("Original: double = "+c.doubleField); System.out.println("Original:
    forField.int = "+c.forField.getInt());
}
for (Cloning c : scArrClone) { System.out.println("Klone:
    double = "+c.doubleField);
    System.out.println("Klone: forField.int = "+c.forField.getInt());
}
```

15.11-rasm. 7-misol

Manba: [4]

Bu yerda massivni klonlagandan so'ng, klonlangan massivning birinchi elementining qiymatlari tuzatiladi. Shundan so'ng asl va klonlangan massivlarning barcha elementlari ajratib ko'rsatiladi. Natijadan ko'rinib turibdiki, asl massiv ham o'zgaradi:

```
Manba: double = 12.12
Manba:      forField.int = 12
Manba: double = 12.12
Manba:      forField.int = 12
Manba: double = 12.12
Manba:      forField.int = 12
Manba: double = 12.12
Manba:      forField.int = 12
Original: double = 36.36
Original: forField.int = 36
Original: double = 36.36
Original: forField.int = 36
Original: double = 36.36
Original: forField.int = 36
```

```
Original: double = 36.36
Original: forField.int = 36
Klon: double = 36.36 Klon:
forField.int = 36 Klon:
double = 36.36 Klon:
forField.int = 36 Klon:
double = 36.36 Klon:
forField.int = 36 Klon:
double = 36.36 Klon:
forField.int = 36
```

Massivlarni klonlashda jismoniy chegarani "bo'ysunish" bo'yicha barcha mas'uliyat massivni o'z ichiga olgan klass zimmasida bo'ladi.

Kloningning yana bir yondashuvi – linearizatsiya orqali amalga oshiriladi. Linearizatsiya paytida obyekt avvalo baytlar massiviga aylantiriladi va keyin yana obyekt sifatida qayta yaratiladi. Natijada hosil bo'lgan obyekt manba obyektining kloni bo'ladi. Obyekti klon qilingan sinf (o'rab turuvchi obyekt) va o'rab turuvchi obyektning fizik chegarasiga kiradigan barcha obyektlarga ega sinflar `Serializable` interfeysini amalga oshirishi kerak. Ushbu interfeys Classni `Cloneable` interfeysi kabi hech qanday metodni amalga oshirishga majburlamaydi, lekin yagona farq shundaki, klonlashda `clone()` metodini ustun qo'yish (override) kerak bo'lsa, linearizatsiya qilganda hech narsani qayta aniqlashga hojat yo'q. 7-misol (5.11-rasm) 5.12-rasmdagi kod yordamida linearizatsiya qilinadi.

```
package Examples;
import java.io.Serializable;

public class ForClone implements Serializable { private
    int intField;
    public ForClone (int i) {
        intField = i;
    }
    public void setInt (int i) {intField = i;}
    public int getInt() {return intField;}
}

package Examples;
import java.io.*;
import java.util.*;

public class Serializing implements Serializable{
    private double doubleField;
    private ForClone forField;
    public Serializing (int i, double d) {
        forField = new ForClone(i);
        doubleField = d;
    }
    // Klonlash vositasi
```



```
public Object clone() { try
{
    // ob'ektni bayt massiviga saqlash
    ByteArrayOutputStream bout = new ByteArrayOutputStream();
    ObjectOutputStream out = new ObjectOutputStream(bout);
    out.writeObject(this);
    out.close();
    // obyektning klonini bayt massividan o'qish
    ByteArrayInputStream bin = new ByteArrayInputStream(bout.toByteArray());
    ObjectInputStream in = new ObjectInputStream(bin);
    Object ret = in.readObject();
    in.close();
    return ret;
} catch (Exception e) {
    return null;
}
}

public static void main (String[] args) throws CloneNotSupportedException { Serializing
    sr = new Serializing(12, 12.12);
    Serializing srNew = (Serializing)sr.clone();
    srNew.doubleField = 24.24;
    srNew.forField.setInt(24);
    System.out.println("Original doubleField = "+sr.doubleField);
    System.out.println("Clone doubleField = "+srNew.doubleField);
    System.out.println("Original forField.intField = "+sr.forField.getInt());
    System.out.println("Clone forField.intField = "+srNew.forField.getInt());
}
}
```

15.12-rasm. 7-misol o'zgartirilgan

Manba: [4]

Natija avvalgi misoldagidek:

```
Asl doubleField = 12.12 Klon
doubleField = 24.24 Asl
forField.intField = 12 Klon
forField.intField = 24
```

15.5. XULOSA

Java uchun murakkab obyektlarni o'chirish avtomatik axlat yig'ish tufayli juda oson. Hech bir ko'rsatkich obyektga ishora qilmay qolganda, obyekt avtomatik ravishda o'chiriladi. Java dasturchisi uchun bunday vazifa umuman yuzaga kelmaydi, deyishimiz mumkin.

ADABIYOTLAR

- [1] Booch G., Maksimchuk R., Engle M., Young B., Conallen j., va Houston K. (2007) *Ob'ektga yo'naltirilgan tahlil va dizayn: ilovalar bilan*. 3-nashr. Addison-Wesley, 720 b.
- [2] Fowler M. (2000) *Standart Ob'ektli Modellash Tili bo'yicha Qisqa Qo'llanma (3-nashr)*. Addison-Wesley, 181 bet.
- [3] Meyer B. (1997) *Ob'ektga yo'naltirilgan dasturiy ta'minotni yaratish*. 2-nashr, Prentice Hall, 770 bet.
- [4] Horstmann C. va Cornell G. (2012) *Core Java*. 9-nashr, Prentice Hall, 1-jild, 2-jild.
- [5] Deitel P. va Deitel H. (2009) *Java™ for Programmers: Deitel® Developer Series*. Prentice Hall, 1200 bet.
- [6] Gilbert S. va McCarty B. (1998) *Java dasturida obyektga yo'naltirilgan dizayn*. SAMS, 731 bet.
- [7] Connolly T. va Begg C. (2002) *Ma'lumotlar bazalari tizimlari: Dizayn, amalga oshirish va boshqaruv bo'yicha amaliy yondashuv*. 3-nashr. Addison-Wesley, 1440 bet.
- [8] Chanyshev A. N. (1981) *Qadimgi falsafa bo'yicha ma'ruzalar kursi: universitetlar falsafa fakultetlari va bo'limlari uchun o'quv qo'llanmasi*. Moskva, Высшая школа (rus tilida).
- [9] <http://www.adobe.com/devnet/actionscript/learning/oop-concepts/composition-and-aggregation.html>
- [10] *JDK hujjatlari*. <http://docs.oracle.com/javase/specs/jls/se8/jls8.pdf>